

Numerical calculus

B

B.1 Differences and sums

We are often interested how a variable, such as the membrane potential or population rate of cell assemblies, change with time. Let us call this quantity $x(t)$ for now, where we indicated that it changes with time. The change of this variable from time t to time $t' = t + \Delta t$ is then

$$\Delta x = x(t + \Delta t) - x(t). \quad (\text{B.1})$$

The quantity Δt is the finite difference in time. For a continuously changing quantity we could also think about the instantaneous change value, dx , by considering an infinitesimally small time step. Formally,

$$dx = \lim_{\Delta t \rightarrow 0} \Delta x = \lim_{\Delta t \rightarrow 0} (x(t + \Delta t) - x(t)). \quad (\text{B.2})$$

The infinitesimally small time step is often written as dt . Calculating with such infinitesimal quantities is covered in the mathematical discipline of calculus, but on the computer we have always finite differences and we need to consider very small time steps to approximate continuous formulation. With discrete time steps, differential become differences and integrals become sums

$$dx \rightarrow \Delta x \quad (\text{B.3})$$

$$\int dx \rightarrow \Delta x \sum \quad (\text{B.4})$$

Note the factor of Δx in front of the summation in the last equation. It is easy to forget this factor when replacing integrals with sums. The following demonstrates this discretization for a differential equation.

B.2 Numerical integration of an initial value problem

The time dynamics of several models in this book are defined with differential equations that specify the change of a quantity dx for an infinitesimal time step dt . This change is specified by a function $f(x, t)$, which may depend on the quantity x and sometimes also explicitly by the time. Mathematically this is expressed by

$$\tau \frac{dx}{dt} = f(x, t), \quad (\text{B.5})$$

B.1 Differences and sums	327
B.2 Numerical integration of an initial value problem	327
B.3 Euler method	328
B.4 Higher-order methods	330
B.5 Adaptive Runge-Kutta	331
Further reading	334

which is a first-order differential equation with one independent variable and time constant τ . We outline here the principles of some numerical integration techniques for this simple case, but the methods can be generalized easily to more complex coupled differential equations and differential equations that contain higher-order differentials. We are concerned with calculating the values of the quantity $x(t)$ for specific times $t > t_0$ when the value of the quantity at time $t = t_0$ is known,

$$x(t_0) = x_0. \quad (\text{B.6})$$

This is called an *initial value problem*. We often refer to the initial value problem in the text as numerical integration. However, numerical integration can also refer to other numerical tasks such as numerically approximating integrals like $\int_{x_1}^{x_2} f(x)dx$, or to finding solutions of differential equations with other constraints.

B.3 Euler method

Digital computers can only represent discrete numbers and not infinitesimally small changes. We must therefore express the continuous dynamic given by eqn B.5 in discrete time steps. We call this procedure *discretization*. The simplest form is to use small but finite time steps between two consecutive times t_1 and t_2 . We write this time step as

$$\Delta t = t_2 - t_1. \quad (\text{B.7})$$

The continuous process is recovered in the limit $\Delta t \rightarrow 0$, which we call the *continuum limit*. The change of the quantity between the two time steps can be expressed as

$$\Delta x(t + \Delta t) = x(t + \Delta t) - x(t). \quad (\text{B.8})$$

The differential eqn B.5 can thus be discretized by writing it as a difference equation

$$\frac{\Delta x(t + \Delta t)}{\Delta t} = f(x(t), t). \quad (\text{B.9})$$

We have assigned the value of the change specified on the right-hand side at time t to the change at time $t + \Delta t$, though we could have also chosen $f(x(t + \Delta t), t)$ as the right hand side. The numerical values would then be a little bit different and the discrete systems are indeed a little bit different. However, we have to make all the following choices in the light of the continuum limit in which we should recover the same answers for the different methods. Substituting eqn B.8 into eqn B.9 gives

$$x(t + \Delta t) = x(t) + \Delta t f(x(t), t). \quad (\text{B.10})$$

This equation simply states the meaning of a differential in a discrete situation. The value of variable x at the next time step is given by the value of the variable at the previous time step times plus Δt the change specified by the function f . This iterative method tracks the development of quantity x from an initial value to later times by summing up the changes. This simplest method of numerical integration, which extrapolates changes linearly, is known as the *Euler method*.

Table B.1
with initial

To illu
tion

The cor
can be s

which c
function
'Exact s

The app
 $\Delta t = 0$
relative
is the a
this app
true val
only slo
step. T
linear c
 Δt is s
in not

Table B.1 Values of different numerical solutions of the differential equation $dx/dt = t - x + 1$ with initial condition $x(0) = 1$

time t	Exact solution	Euler method	Runge-Kutta (adaptive)
0	1	1	1
0.2	1.01873075	1.0400	1.01873077
0.4	1.07032005	1.1120	1.07032008
0.6	1.14881164	1.2096	1.14881168
0.8	1.24932896	1.3277	1.24932901
1.0	1.36787944	1.4621	1.36787949
1.2	1.50119421	1.6097	1.50119426
1.4	1.64659696	1.7678	1.64659701
1.6	1.80189652	1.9342	1.80189656
1.8	1.96529889	2.1074	1.96529893
2.0	2.13533528	2.2859	2.13533532

To illustrate this procedure we follow a specific example, given by the function

$$f(x, t) = 1 - x + t. \quad (\text{B.11})$$

The corresponding differential equation (inserting the function into eqn B.5) can be solved analytically. With the initial condition $x(0) = 1$, this is given by

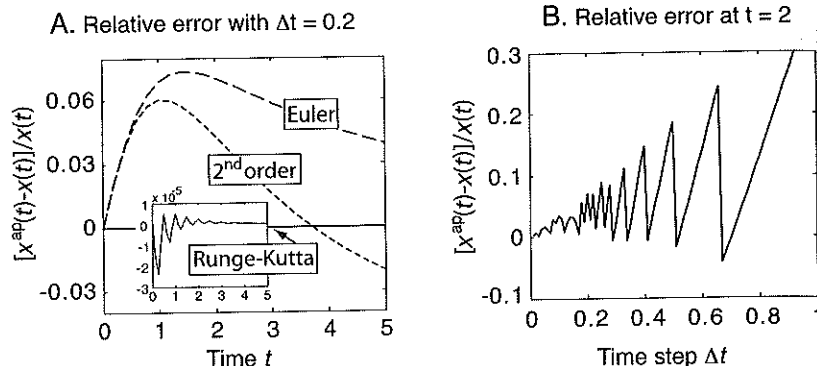
$$x(t) = t + e^{-t}, \quad (\text{B.12})$$

which can be verified by inserting this function into eqn B.5. Values for this function at different time steps are shown in Table B.1 in the column labelled 'Exact solution'. The Euler method for this example is

$$x(t + \Delta t) = x(t) + (1 - x + t)\Delta t. \quad (\text{B.13})$$

The approximation values calculated with the Euler method using a time step of $\Delta t = 0.2$ are included in the column labelled 'Euler method' in Table B.1. The relative difference between these two columns, $[x^{\text{ap}}(t) - x(t)]/x(t)$, where x^{ap} is the approximation from different numerical integration methods discussed in this appendix, is plotted in Fig. B.1A. The Euler method is overestimating the true value in this particular example since the slope of the solution is increasing only slowly and we overestimated the slope by taking the finite integration time step. The relative error is decreasing with time since the function becomes more linear on larger scales. The error at $t = 2$ for different integration time steps Δt is shown in Fig. B.1B. This is a non-monotone function so that tuning Δt is not always simple.

Fig. B.1 (A) The relative difference between numerical solutions and the solution for the differential equation $dx/dt = t - x + 1$ with the initial condition $x(0) = 1$. For the Euler method and the second order method we set the integration time step to $\Delta t = 0.2$. The higher order Runge-Kutta method uses an adaptive integration time step. The absolute error is much smaller than in the other cases, and differences less than $O(10^{-5})$ are depicted in the insert. (B) Relative error of the Euler method for different values of the integration time step Δt at time $t = 2$.



B.4 Higher-order methods

In the Euler method just outlined we have only taken the first derivative of the function f (the slope of the function) into account and have used this to extrapolate linearly to the value of x at the next time step $t + \Delta t$. We can improve the accuracy of the approximation if we also take into account higher-order derivatives (curvature terms). We can formalize this by considering the *Taylor expansion* of the function around the time for which we want to estimate the value. The Taylor expansion is given by

$$x(t + \Delta t) = x(t) + \Delta t \frac{dx}{dt} + \frac{1}{2}(\Delta t)^2 \frac{d^2x}{dt^2} + O((\Delta t)^3), \quad (\text{B.14})$$

where the last term stands for all possible higher-order terms. This equation with the first two terms is the Euler method (eqn B.10) since the derivative is given by $f(x, t)$ as stated by eqn B.5. If we consider the next order term in the Taylor expansion for our example, we get the approximation

$$x(t + \Delta t) = x(t) + (1 - x + t)\Delta t + \frac{1}{2}(1 - x)(\Delta t)^2, \quad (\text{B.15})$$

which adds a term proportional to $(\Delta t)^2$ to the previous expression. The second order relative difference of the second order solution to the exact solution is also shown in Fig. B.1A. While this method is a little bit better for this example for intermediate extrapolation times, the error increases again for large times.

If we use higher-order terms we can further improve the approximations of the exact solution. This method requires that we know higher derivatives of the function f explicitly. In case they are difficult to calculate analytically, we can also estimate them numerically, although this introduces further numerical errors. This method is therefore not used directly but rather indirectly in the following way. If we use only the first derivative (slope) of the function at time t we assume that this is the same for subsequent times. If this is changing (for example, if the function has a curvature) then we use the slope at a time between t and $t + \Delta t$ as a better guess for the average slope in this interval. Thus, instead of using $f(x, t)$ in eqn B.10 we should use

$$f(x, t) \rightarrow f(\bar{x}, t + \alpha \Delta t), \quad (\text{B.16})$$

where α is, $\bar{x} =$ this using

A comm is called and con requires we mad Therefo solution

B.5

It is no and it proximi is there method

which drew f better

App results the tim do not when and ha strateg in the algorit our ex Runge Table the st better The ferent

where α is a parameter. The value $\tilde{x}$ should be the value of x at $t + \alpha\Delta t$, that is, $\tilde{x} = x(t + \alpha\Delta t)$, which we don't know *a priori*. We could, however, guess this using the first-order Euler method, that is,

$$\tilde{x} = x(t) + \alpha\Delta t f(x(t), t). \quad (\text{B.17})$$

A common choice for the parameter α is $\alpha = 1/2$, in which case this method is called the *midpoint method*. This method cancels error terms of first order and corresponds therefore to a second-order method. The midpoint method requires two function calls, but no higher derivatives are needed. Note that we made several assumptions that might often be appropriate, but not always. Therefore, higher-order methods can usually improve the approximation of the solution $x(t)$, but this does not have to be the case in all circumstances.

B.5 Adaptive Runge-Kutta

It is not necessary to stop at the second-order and midpoint approximation and it is possible (though tedious) to work out solutions for higher-order approximations. The method most commonly used for numerical integrations is thereby a fourth-order method known as the *Runge-Kutta method*. This method is given by the following set of equations

$$k_1 = \Delta t f(x(t), t) \quad (\text{B.18})$$

$$k_2 = \Delta t f\left(x(t) + \frac{1}{2}k_1, t + \frac{1}{2}\Delta t\right) \quad (\text{B.19})$$

$$k_3 = \Delta t f\left(x(t) + \frac{1}{2}k_2, t + \frac{1}{2}\Delta t\right) \quad (\text{B.20})$$

$$k_4 = \Delta t f(x(t) + k_3, t + \Delta t) \quad (\text{B.21})$$

$$x(t + \Delta t) = x(t) + \frac{1}{6}k_1 + \frac{1}{3}k_2 + \frac{1}{3}k_3 + \frac{1}{6}k_4, \quad (\text{B.22})$$

which is a direct generalization of the midpoint method. The conclusions we drew for the midpoint method still hold. This fourth-order method is often better than lower-order methods, although this cannot always be guaranteed.

Applying any of these numerical methods requires that we check that the results we get do not critically depend on the choice of the parameters such as the time step Δt . At the very least we should check that the numerical results do not change more than within a certain range that we demand of the solutions when modifying the time step parameter considerably (for example, double it and halve it). However, many numerical solvers have already implemented such strategies with algorithms that change the time step as long as the variations in the results are lower than the error bound we give to the system. Such algorithms are called *adaptive time step methods*. The numerical solution of our example solved with the MATLAB routine `ode45()`, which is basically a Runge-Kutta of at least fourth order with adaptive time steps, is included in Table B.1. The maximum of the absolute difference is less than 5×10^{-8} with the standard parameters used by MATLAB. This is six orders of magnitude better than the Euler approximation.

There are several other methods for numerical integration, each having different strengths and weaknesses in certain application domains. For example,

Table B.2 Some MATLAB functions for numerical integrations of first-order differential equations [adapted from Hanselman and Littlefield, *Mastering MATLAB® 5*, Prentice Hall (1998)]

MATLAB function	Method	Comments
ode23	Runge-Kutta (order 2-3)	For problems that are non-stiff or that are discontinuous, or where lower accuracy is acceptable
ode45	Runge-Kutta (order 4-5)	Non-stiff problems, often tried first for new problem
ode15s	Multistep (order 1-5)	Stiff problems, try if ode45 fails or is too inefficient
ode113	Adams-Bashford-Moulton (order 1-13)	Non-stiff-problems, multistep method, where $f(x, t)$ is expensive to compute, not suitable for discontinuities
ode23s	Modified Rosenbrock (order 2)	Stiff problems with discontinuities, lower accuracy is acceptable

the Runge-Kutta method breaks down if there are very sudden changes in the exact solution for which the linear interpolations used in the midpoint approach break down. MATLAB provides implementations of several other solvers that should be considered for various types of problems. A summary of such solvers is provided in Table B.2. The different solvers are provided within MATLAB with the same generic function calls, so that a switch between solvers can easily be achieved by changing only the name of the function.

Simulation

The results shown in Fig. B.1 are produced with the program in Table B.3. In Line 1 we make sure to clear the workspace (`clear`) and possible figure (`clf`), and hold the plot so that several lines can be drawn in the same figure. The numerical integrations with the Euler method (Lines 3-7) and the second-order method (Lines 8-12) are direct translations of the formulas above. In Line 15 we use the MATLAB function `ode45()`, which is an efficient implementation of an adaptive higher-order Runge-Kutta method. The right-hand side of the differential equation is thereby provided in a function we have to write. In this example we use in Line 14 a MATLAB construct called *anonymous function* to define this function in the same file. Lines 16-20 are then used to generate a plot of results used in Fig. B.1A.

The results in Fig. B.1B are produced by the rest of the code. It starts by calling `figure`, which produces first a new figure window so that the previous plot is not overwritten. We then run the Euler method many times with different time steps Δt .

Table B.3 Program app.numinte.m

```

1  clear; clf; hold on;
2  dt=0.2; tend=5;
3  %% Euler method
4  x=1; xEuler=x;
5  for t=dt:dt:tend
6      x=x+dt*(1-x+t); xEuler=[xEuler,x];
7  end
8  %% Second order
9  x=1; xSecond=x;
10 for t=dt:dt:tend
11     x=x+dt*(1-x+t)+dt^2*(1-x)/2; xSecond=[xSecond,x];
12 end
13 %% Runge-Kutta
14 ode_function= @(t,x,flag) 1-x+t;
15 [t,xRK]=ode45(ode_function,[0 tend],1,[]);
16 %% Plotting results
17 x=t+exp(-t); plot(t,(xRK-x)./x,'r');
18 t=0:dt:tend; x=t+exp(-t);
19 plot(t,(xEuler-x)./x);
20 plot(t,(xSecond-x)./x,'g');
21 %% Plot dt error
22 figure; xDIFF=[];
23 for dt=0.01:0.01:1
24     x=1; for t=dt:dt:2; x=x+dt*(1-x+t); end
25     xDIFF=[xDIFF,(x-2-exp(-2))/(2+exp(-2))];
26 end
27 dt=0.01:0.01:1; plot(dt,xDIFF);

```

Further reading

The book by Press *et al.* (1992) is a standard reference you will find on the desks of most people applying computer simulations. The book by Flower (1995) is a book that is more teaching-oriented with clear descriptions and explicit program examples.

William H. Press, Saul A. Teukolsky, William

T. Vetterling, and Brian P. Flannery (1992), *Numerical recipes in C: The art of scientific computing*, Cambridge University Press, 2nd edition.

B. H. Flowers (1995), *An introduction to numerical methods in C++*, Oxford University Press.

Basics

This appendix covers the basic concepts of probability density functions. It is primarily for those who are new to the concepts of probability.

C.1 Random Variables

The most common type of random variable is a quantity that can take on a range of values. Such as in the case of a regular variable, the value of the random variable is determined by a mathematical function. This is often useful as it allows us to describe the frequency of a particular value since a random variable can be described by a frequency distribution. We can take into account the probability of a particular value of a random variable.

We can describe a random variable in terms of its possible values. This is often done by listing the possible values and their corresponding probabilities. This is often done by listing the possible values and their corresponding probabilities. This is often done by listing the possible values and their corresponding probabilities.

describes the probability of a particular value of X occurring. This is often done by listing the possible values and their corresponding probabilities. This is often done by listing the possible values and their corresponding probabilities.

Fundamentals of Computational Neuroscience

SECOND EDITION

Thomas P. Trappenberg

Dalhousie University

OXFORD
UNIVERSITY PRESS

VIRGINIA COMMONWEALTH UNIVERSITY